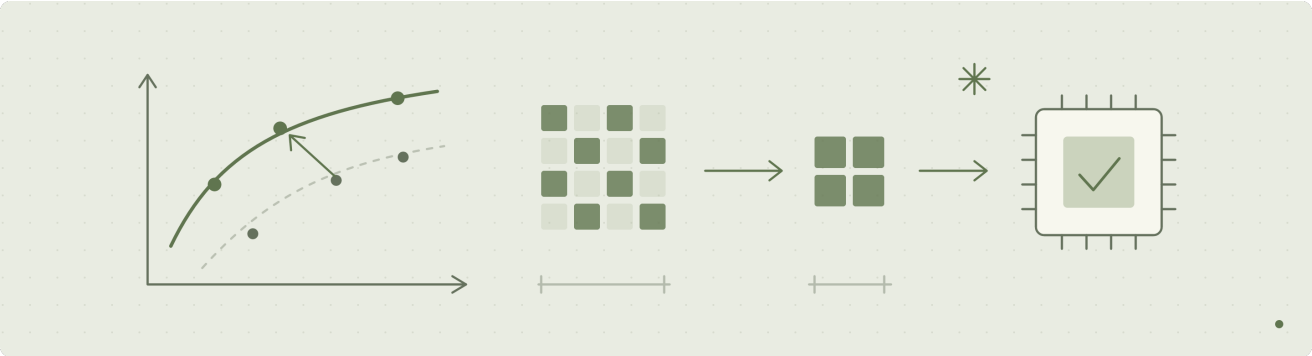
Lecture 1. Introduction

© 2026 Alexander Petrov. Study notes by Alexander Petrov.

Original publication: https://petrovortex.github.io/library/efficient-ml/lecture-1-introduction/

Based on Efficient ML by Song Han · MIT 6.5940. Original course materials belong to their respective authors.

Course: https://efficientml.ai/



In This Lecture

1. **Motivation:** Algorithmic scaling laws demand exponential growth in model size, while silicon physics (Moore's Law & Dennard Scaling) has hit a wall in memory capacity and memory bandwidth.
2. **Objective:** Because we cannot solve this by waiting for better general-purpose hardware, we must reformulate deep learning as a **constrained multi-objective optimization problem** (Accuracy vs. Physical Cost).
3. **Decomposing the Cost Function:** When we write down the exact equation for executing a neural network layer on silicon, we discover there are only **three mathematical variables** contributing to physical cost: **Weights (W)**, **Activations (X)**, and **Graph Topology (f)**.
4. **The Solutions Across Modalities:** Every single paper and demo shown in the lecture (Vision, LLMs, Multimodal, On-Device Training) is not a random trick - it is what happens when you push one of those three variables (W, X, f) to its physical limit across different hardware regimes.
5. **Algorithm-System Co-Design:** Finally, reducing theoretical mathematical operations does not automatically make code run faster on real chips. We must co-design our algorithms with the physical instruction sets and memory hierarchies of GPUs, NPUs, and Microcontrollers.

Part 1: Motivation

Modern deep learning is governed by empirical scaling laws: model capability (such as emergent zero-shot reasoning and Chain-of-Thought in LLMs[1], or high-fidelity video generation) is a monotonic function of parameter count and token compute. Between 2012 (AlexNet[2], 6×10^7 parameters) and 2026 (frontier models at 2.7×10^{12} to 10^{13} parameters), model sizes grew by **5 orders of magnitude**.

However, the hardware executing these models is governed by semiconductor physics:

1. **Memory Growth Not Fast Enough:** GPU High Bandwidth Memory (HBM) grew from 16 GB (P100) $\rightarrow$ 32 GB (V100) $\rightarrow$ 80 GB (A100) $\rightarrow$ 192 GB (B200) $\rightarrow$ >200 GB (Rubin). The slope of model demand is dramatically steeper than the slope of hardware supply.
2. **The End of Dennard Scaling & Single-Thread Performance:** Looking at 40 years of microprocessor data (1970-2020), transistor counts continue to grow, but clock frequency, single-thread performance, and Thermal Design Power (TDP, watts per chip) have plateaued.
3. **The Data Movement Bottleneck:** In silicon, **arithmetic is cheap; moving data is expensive**. Fetching a 32-bit weight from off-chip DRAM to on-chip SRAM registers consumes orders of magnitude more energy and latency than performing the actual multiplication on the ALU.

If we cannot rely on hardware clock speeds or DRAM capacity to catch up with model sizes, we can no longer treat accuracy as an unconstrained optimization problem. We must bring physical hardware cost directly into our mathematical objective.

Part 2: Formalizing the Objective

In standard ML courses, you minimize loss $\mathcal{L}(\theta)$ to maximize accuracy. In Efficient ML, every model exists in a 2D space: **Accuracy** (y-axis) vs. **Physical Resource Cost** (x-axis).

To quantify "Physical Resource Cost" before touching specific hardware, we measure two fundamental quantities:

1. **Compute Volume - MACs (Multiply-Accumulate Operations):** The fundamental primitive of linear algebra in neural networks is $a \leftarrow a + w \cdot x$. Each MAC equals 2 FLOPs.
2. **Memory Footprint & Traffic (Bytes):** The number of bits required to store parameters W (static memory) and intermediate activations X (dynamic runtime memory), plus the bandwidth required to stream them between memory tiers.

When you scale up a standard architecture (e.g., ResNet-18 $\rightarrow$ ResNet-152[3], or LLaMA-7B $\rightarrow$ LLaMA-70B[4]), you move toward the top-right of the Accuracy-MACs plane with severe diminishing returns.

Our goal in this course is to move to the **top-left corner** - so that we achieve the same accuracy with $10\times$-$50\times$ less compute and memory.

How do we mathematically move a neural network toward the top-left corner? To answer that, we must look at what a neural network actually computes and identify the exact mathematical levers we are allowed to pull.

Part 3: Decomposing the Cost

Almost every layer in a deep neural network - whether a ConvNet, a Linear layer in an LLM, or a Diffusion U-Net[5]/DiT[6] - reduces to a parameterized tensor contraction over a graph of L layers: $X^{(l+1)} = f_l\left(W^{(l)}, X^{(l)}\right)$

If we want to reduce the memory and MACs of this equation without destroying its representational capacity, what are our degrees of freedom? There are **only three**:

1. **Lever 1: Compress the Weights - $W^{(l)}$**
   - *Pruning / Sparsity:* Set redundant entries (or entire channels) of W to zero so they don't need to be stored or multiplied ($\|W\|_0 \ll \dim(W)$).
   - *Quantization:* Reduce the bit-width b_W used to represent each scalar in W (e.g., from 32-bit/16-bit floats down to 8-bit or 4-bit integers).
2. **Lever 2: Compress the Activations / Tokens - $X^{(l)}$**
   - *Token / Spatial Sparsity:* If X has N spatial pixels or sequence tokens, not all N tokens carry information. Drop redundant tokens or pixels dynamically.
   - *Activation Quantization:* Reduce the bit-width b_X of $X^{(l)}$ so intermediate buffers fit in tiny SRAM caches and can use fast integer ALUs.
3. **Lever 3: Redesign the Operator Topology (f_l)**
   - *Neural Architecture Search (NAS) & Operator Replacement:*[7] Replace mathematically heavy operators - such as $O(N^2)$ softmax self-attention[8] - with $O(N)$ linear attention, hardware-friendly convolutions, or weight-shared sub-networks (Once-for-All[9] / Anycost networks[10]).

Now that we have the 3 levers (W, X, f), why did Professor Han spend 40 minutes showing dozens of demos across Vision, Language, and Robotics? Because **different modalities and tasks bottleneck on different levers**. Let's trace how each domain forces us to use a specific combination of W, X, and f.

Part 4: How Different Modalities Bottleneck on W, X, and f

In Part 3, we reduced layer execution to $X^{(l+1)} = f_l(W^{(l)}, X^{(l)})$. Why does the lecture survey so many different applications across vision, language, and robotics? Because the dominant bottleneck in that equation shifts completely depending on the data modality and the target hardware.

1. Edge Vision and Microcontrollers (SRAM Capacity Limit)

A microcontroller like the ARM Cortex-M7 operates on a few hundred milliwatts, has no operating system or virtual memory, and provides only ~256 KB of read/write SRAM alongside 1 MB of read-only Flash. Standard vision models require hundreds of megabytes.

For **inference**, static weights W can stay in read-only Flash, while dynamic feature maps $X^{(l)}$ must fit inside the 256 KB SRAM. **MCUNet**[11] solves this by jointly optimizing the operator graph f (via Neural Architecture Search) and quantizing both W and X to 8-bit integers, allowing real-time person and mask detection on a microcontroller.

On-device training introduces a much stricter mathematical bottleneck. First, backpropagation requires roughly $3\times$ the compute of inference (one forward pass, one backward pass for activation gradients, and one for weight gradients). Second, consider the weight gradient equation for layer l:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \left(X^{(l)}\right)^T \frac{\partial \mathcal{L}}{\partial X^{(l+1)}}$$

During inference, once $X^{(l+1)}$ is computed, the buffer for $X^{(l)}$ can be overwritten immediately. During training, however, the chain rule forces the device to keep all intermediate forward activations $\{X^{(1)}, X^{(2)}, \ldots, X^{(L)}\}$ in SRAM simultaneously until the backward pass reaches each respective layer.

To compress training memory from over 600 MB down to **141 KB** (a $> 4000\times$ reduction), Han Lab combined three techniques[12]:

- **Sparse Layer and Sparse Tensor Updates:** If we freeze a layer or a subset of channels during the backward pass ($\Delta W^{(l)} = 0$), we no longer need its forward activation $X^{(l)}$ in the gradient equation above, so we simply do not store it during the forward pass.
- **Quantization-Aware Scaling (QAS):** Performing weight updates directly in 8-bit integer precision fails because gradients and weights have mismatched scales across layers. QAS analytically rescales the gradient norms so optimization converges in 8-bit without keeping 32-bit master weights in memory.
- **Tiny Training Engine (TTE):** Standard frameworks like PyTorch[13] build dynamic autograd graphs at runtime, wasting precious SRAM on graph metadata. TTE moves graph pruning and backward scheduling to compile time, deploying only stripped C code to the microcontroller.

2. High-Resolution, Generative, and 3D Vision (Token Dimension N and Operator Complexity f)

When we move from classifying small images to high-resolution segmentation, image/video diffusion, and 3D perception, parameter storage W is no longer the primary problem. Instead, the spatial or temporal dimension N of the activation tensor X explodes.

If a standard classification ViT[14] requires $1\times$ MACs, an 8B LLaMA forward pass takes roughly $10\times$, **Flux**[15] (image generation) takes $36\times$, and **CogVideoX**[16] (video generation) takes over $300\times$. In high-resolution vision and video, N grows quadratically with image resolution and linearly with frame count, while standard self-attention scales as $O(N^2)$ and diffusion models iterate the entire network dozens of times per sample.

Because the bottleneck here lies in X and f, compression targets the operator topology and spatial redundancy:

- **Replacing $O(N^2)$ Operators (EfficientViT-SAM):**[17] In the Segment Anything Model (SAM)[18], high-resolution image tokens make standard softmax attention prohibitively slow (~11 FPS). Replacing

quadratic softmax attention with hardware-efficient linear attention ($O(N)$) accelerates throughput up to $48\times$ (182-500 FPS) at the exact same segmentation accuracy.

- **Elastic Sub-networks (Anycost GAN):**[10] During interactive image editing, a user dragging a slider to change a facial expression does not need final photorealistic fidelity on every intermediate frame. By training a single weight-shared "supernet" that contains smaller callable subsets of channels, the system runs a $5.2\times$ cheaper sub-network in real time while the user drags the slider, executing the full model only when the user clicks "Finalize".
- **Exploiting Spatial Sparsity (SIGE and BEVFusion):**[19] In image inpainting (generating a horse inside a small masked box on a grassland), recomputing unchanged background pixels wastes compute. Updating only the active masked activations yields a $4.8\times$ speedup. Similarly, in autonomous driving (**BEVFusion**)[20], 3D LiDAR point clouds are naturally sparse while multi-camera RGB images are dense. Fusing both into a unified Bird's-Eye-View representation with sparse operators increases perception speed from 5 FPS to 47 FPS on a single mobile GPU.
- **Distributed Inference:**[21] Splitting image generation across 4 GPUs cuts compute time from 12 seconds to 3 seconds, but lack of synchronization creates visible seams between image patches. Communicating compressed activation features across GPUs removes the visual artifacts and finishes generation in 4 seconds.

3. Large Language Models and Multimodal VLMs (Memory Bandwidth and Activation Outliers)

Large Language Models - and Vision-Language Models like **VILA**[22], which project images and video into visual tokens alongside text tokens - exhibit an entirely different hardware bottleneck.

Capabilities like zero-shot generalization, in-context learning, and Chain-of-Thought reasoning only emerge clearly once models cross a critical parameter threshold (tens of billions of parameters). Furthermore, Chain-of-Thought reasoning solves complex problems by generating long sequences of intermediate tokens.

Because autoregressive generation produces **one token at a time** sequentially, computing a single new token requires multiplying a tiny activation vector x (batch size 1) against the entire multi-gigabyte weight matrix W. The ALU finishes the arithmetic almost instantly and spends the rest of the cycle stalled, waiting for W and the growing KV-cache to stream from off-chip DRAM. Therefore, LLM decoding is strictly **memory-bandwidth bound**.

To accelerate LLMs and fit them onto laptops or robotics controllers (such as Han Lab's 15 Hz ping-pong and coffee-making World Action Models[23]), we must shrink both the KV-cache (X) and the weight traffic (W):

- **Token Pruning (SpAtten):**[24] Natural language contains massive semantic redundancy. By inspecting attention scores on the fly, SpAtten drops uninformative tokens progressively across layers (for instance, reducing *"As a visual treat, the film is almost perfect"* to just *"film perfect"*), directly cutting KV-cache memory traffic.

- **8-Bit Weight and Activation Quantization (SmoothQuant):**[25] To use fast 8-bit integer instructions (W8A8), we must quantize both W and X. However, LLMs above ~6.7B parameters develop systematic **activation outliers**: a few fixed channels in X consistently exhibit magnitudes $100\times$ larger than all other channels. If you apply standard per-tensor quantization to X, those few outliers stretch the quantization step size across the entire range, rounding all normal activation values to zero. SmoothQuant resolves this using a mathematical invariance:

  $$Y = XW = (XS^{-1})(SW) = \hat{X}\hat{W}$$

  By choosing a diagonal scaling matrix S that divides the outlier channels of X and multiplies the corresponding rows of W, the outlier variance is migrated from the activations into the weights offline. Both $\hat{X}$ and $\hat{W}$ become well-balanced and can be quantized to INT8 without accuracy loss.

- **4-Bit Weight-Only Quantization (AWQ):**[26] When deploying an LLM or VLM locally for single-user generation (W4A16), memory bandwidth of the weights is the primary bottleneck. Naive rounding of weights to 4-bit integers destroys model reasoning. **Activation-Aware Weight Quantization (AWQ)** observes that not all weights contribute equally to the output error $\|WX - Q(W)X\|$: the 1% of weight channels that multiply the large activation channels dominate the layer's output. By scaling up just those 1% salient weight channels prior to 4-bit quantization, AWQ preserves full-precision quality across LLMs and VLMs at a $4\times$ smaller memory footprint.

Part 5: Algorithm-System Co-Design

Suppose you implement AWQ in PyTorch to quantize a 7B model's weights to 4-bit integers, or you prune 75% of a network's weights to zero. If you run that model using standard PyTorch layers, your wall-clock inference speed will not improve - in fact, it will likely run slower. **Theoretical reduction in MACs or bit-width does not automatically translate into physical speedup.**

1. The Byte-Alignment Gap (Why TinyChat Engine Exists)

Modern CPU and GPU memory architectures are **byte-aligned**: the smallest addressable unit of memory in C/C++ and hardware load/store instructions is 1 byte (8 bits).

If you quantize weights to 4 bits (half a byte), a standard software array still stores each 4-bit number inside an 8-bit int8 container, wasting half the memory and saving zero bandwidth. Furthermore, hardware Tensor Cores cannot multiply 4-bit weights against 16-bit activations natively without alignment conversion.

Bridging this gap requires co-designing the algorithm (**Lab 4: AWQ**) with the low-level runtime engine (**Lab 5: TinyChat Engine**[27]). To turn a $4\times$ theoretical compression into a $> 3\times$ measured wall-clock speedup (e.g., jumping from 50 tokens/s to 166 tokens/s on an RTX 4090, or running at 30 tokens/s on a Jetson Orin), the system engine must:

1 Pack two 4-bit weights into a single 8-bit byte in DRAM so memory transfers are genuinely reduced by $4\times$ compared to FP16.

2 Load the packed bytes into on-chip registers and execute custom bitwise shift and mask operations (>>, &) fused directly inside the matrix-multiplication kernel, unpacking them into FP16 on the fly right before the FMA instruction executes.

2. Hardware Evolution and Deployment Tiers

Over an 8-year span, single-chip GPU inference throughput increased by more than $300\times$. Because clock speeds and chip power envelopes have plateaued, hardware vendors achieved this gain by hardwiring silicon support for the exact algorithmic primitives covered in this course: specialized matrix instructions (DP4A for 8-bit integer dot products, HMMA Tensor Cores for low-precision matrices) and hardware multiplexers for **2:4 Structured Sparsity** (skipping two zeros out of every four weights at the register level). As semiconductor processes shrink from 65nm down to 5nm and below, the software stack accounts for an increasingly dominant share of total engineering cost.

Throughout this course, our software optimizations target three distinct hardware tiers:

Tier	Representative Platforms	Memory Capacity	Memory Bandwidth	Power Envelope
Cloud AI	NVIDIA P100 $\longrightarrow$ A100 $\longrightarrow$ H100 $\longrightarrow$ B200 $\longrightarrow$ Rubin	16 GB $\longrightarrow$ >200 GB HBM	0.7 TB/s $\longrightarrow$ 22 TB/s	250 W $\longrightarrow$ >1,000 W
Mobile / Edge AI	NVIDIA Jetson Orin/Thor, Qualcomm Snapdragon DSP, Apple Neural Engine	4 GB $\longrightarrow$ 128 GB Shared LPDDR	50 GB/s $\longrightarrow$ 273 GB/s	6 W $\longrightarrow$ 100 W
Tiny AI (MCU)	ARM Cortex-M7 (Embedded Microcontrollers)	256-512 KB SRAM, 1 MB Flash	Internal SRAM Bus	0.1 W $\longrightarrow$ 0.3 W

Course Structure and Next Steps

Summary of Lecture 1

1 Deep learning scaling laws are colliding with physical limits in semiconductor memory capacity, memory bandwidth, and power consumption.

2 Efficient ML resolves this by shifting the Accuracy vs. Resource Pareto frontier through three mathematical degrees of freedom: **Weights (W)**, **Activations (X)**, and **Operator Topology (f)**.

3 Realizing physical speedups from these mathematical reductions requires **Algorithm-System Co-Design** - matching sparsity patterns, bit-packing, and operator graphs to the memory hierarchy and instruction sets of the target chip.

Course Roadmap

The remainder of MIT 6.5940 systematically develops each lever introduced today across three modules and five graded practical labs:

- **Foundations & Profiling:**
- *Next Lecture (Lecture 2):* PyTorch Basics, computational graphs, and hardware profiling (**Lab 0** and **Lab 1: GPU Basics & Counting FLOPs/Memory**).
- **Module 1 - Efficient Inference:**
- *Pruning and Sparsity:* Fine-grained vs. structured pruning, lottery ticket hypothesis[28], and 2:4 hardware sparsity (**Lab 2: Pruning**).
- *Quantization:* Post-training quantization (PTQ), Quantization-Aware Training (QAT), SmoothQuant, AWQ, and low-bit formats (**Lab 4: LLM Quantization** and **Lab 5: TinyChat Deployment on Laptop**).
- *Neural Architecture Search (NAS) & Distillation:* Hardware-aware NAS, Once-for-All[9] weight-shared supernets, and teacher-student knowledge transfer[29] (**Lab 3: NAS**).
- **Module 2 - Efficient Training:**
- Distributed training parallelism, gradient compression, on-device training under 256 KB, and federated learning.
- **Module 3 - Application-Specific Optimizations:**
- Deep dives into serving Large Language Models (KV-cache optimization, speculative decoding, sparse attention), accelerating Diffusion/Video models, and fast 3D Point Cloud processing.

References

1 Jason Wei et al.. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. 2022. [back]

2 Alex Krizhevsky et al.. ImageNet Classification with Deep Convolutional Neural Networks. 2012. [back]

3 Kaiming He et al.. Deep Residual Learning for Image Recognition. 2015. [back]

4 Hugo Touvron et al.. Llama 2: Open Foundation and Fine-Tuned Chat Models. 2023. [back]

5 Olaf Ronneberger et al.. U-Net: Convolutional Networks for Biomedical Image Segmentation. 2015. [back]

6 William Peebles and Saining Xie. Scalable Diffusion Models with Transformers. 2022. [back]

7 Barret Zoph and Quoc V. Le. Neural Architecture Search with Reinforcement Learning. 2016. [back]

8 Ashish Vaswani et al.. Attention Is All You Need. 2017. [back]

9 Han Cai et al.. Once-for-All: Train One Network and Specialize it for Efficient Deployment. 2019. [back] [back]

10 Ji Lin et al.. Anycost GANs for Interactive Image Synthesis and Editing. 2021. [back] [back]

11 Ji Lin et al.. MCUNet: Tiny Deep Learning on IoT Devices. 2020. [back]

12 Ji Lin et al.. On-Device Training Under 256KB Memory. 2022. [back]

13 Adam Paszke et al.. PyTorch: An Imperative Style, High-Performance Deep Learning Library. 2019. [back]

14 Alexey Dosovitskiy et al.. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. 2020. [back]

15 Black Forest Labs. Announcing Black Forest Labs (FLUX.1). 2024. [back]

16 Zhuoyi Yang et al.. CogVideoX: Text-to-Video Diffusion Models with An Expert Transformer. 2024. [back]

17 Zhuoyang Zhang et al.. EfficientViT-SAM: Accelerated Segment Anything Model Without Accuracy Loss. 2024. [back]

18 Alexander Kirillov et al.. Segment Anything. 2023. [back]

19 Muyang Li et al.. Efficient Spatially Sparse Inference for Conditional GANs and Diffusion Models. 2022. [back]

20 Zhijian Liu et al.. BEVFusion: Multi-Task Multi-Sensor Fusion with Unified Bird's-Eye View Representation. 2022. [back]

21 Muyang Li et al.. DistriFusion: Distributed Parallel Inference for High-Resolution Diffusion Models. 2024. [back]

22 Ji Lin et al.. VILA: On Pre-training for Visual Language Models. 2023. [back]

23 Song Han. Efficient ML - Lecture 1: Introduction. [back]

24 Hanrui Wang et al.. SpAtten: Efficient Sparse Attention Architecture with Cascade Token and Head Pruning. 2020. [back]

25 Guangxuan Xiao et al.. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. 2022. [back]

26 Ji Lin et al.. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. 2023. [back]

27 MIT HAN Lab. TinyChat: AWQ inference engine and implementation. [back]

28 Jonathan Frankle and Michael Carbin. The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks. 2018. [back]

29 Geoffrey Hinton et al.. Distilling the Knowledge in a Neural Network. 2015. [back]